

Name:Manav Mangesh Uttekar

Roll no.: 71

Problem statement: Implement all the functions of a dictionary (ADT) using hashing and handle collisions using chaining with / without replacement.Data: Set of (key, value) pairs, Keys are mapped to values, Keys must be comparable, Keys must be unique. Standard Operations: Insert(key, value), Find(key), Delete(key)

```
def simple_hash(key, size):
    return sum(ord(c) for c in key) % size

class def simple_hash(key, size):

    return sum(ord(c) for c in key) % size

class Dictionary:

    def __init__(self, size, mode="with"):

        self.size = size

        self.mode = mode # "with" or "without"

        self.table = [[] for _ in range(size)]

    def insert(self, key, value):

        index = simple_hash(key, self.size)

        for i in range(len(self.table[index])):

            if self.table[index][i][0] == key:

                if self.mode == "with":

                    self.table[index][i] = (key, value)

                    print(f"Updated key '{key}' with new value.")

                else:

                    print(f"Key '{key}' already exists. Skipping insert
(without replacement).")

            return

        self.table[index].append((key, value))

        print(f"Inserted key '{key}' successfully.")

    def find(self, key):

        index = simple_hash(key, self.size)

        for k, v in self.table[index]:

            if k == key:

                print(f"Found: {key} => {v}")
```

```

        return v

    print(f"Key '{key}' not found.")

    return None

def delete(self, key):

    index = simple_hash(key, self.size)

    for i in range(len(self.table[index])):

        if self.table[index][i][0] == key:

            del self.table[index][i]

            print(f"Deleted key '{key}'.")

            return True

    print(f"Key '{key}' not found for deletion.")

    return False


def display(self):

    print("\nCurrent state of the dictionary:")

    for i in range(self.size):

        print(f"Index {i}: {self.table[i]}")

    print()


def main():

    print("=== Dictionary ADT using Hashing with Chaining ===")

    size = int(input("Enter size of the hash table: "))

    print("\nChoose collision handling mode:")

    print("1. With replacement (updates existing keys)")

    print("2. Without replacement (ignores duplicate keys)")

    mode_choice = input("Enter 1 or 2: ")

    if mode_choice == "1":

        mode = "with"

```

```
elif mode_choice == "2":
    mode = "without"
else:
    print("Invalid choice. Defaulting to 'with replacement'.")
    mode = "with"

d = Dictionary(size, mode)

while True:
    print("\nOptions:")
    print("1. Insert (key, value)")
    print("2. Find (key)")
    print("3. Delete (key)")
    print("4. Display Dictionary")
    print("5. Exit")

    choice = input("Enter your choice (1-5): ")

    if choice == "1":
        key = input("Enter key: ")
        value = input("Enter value: ")
        d.insert(key, value)
    elif choice == "2":
        key = input("Enter key to find: ")
        d.find(key)
    elif choice == "3":
        key = input("Enter key to delete: ")
        d.delete(key)
    elif choice == "4":
        d.display()
```

```
elif choice == "5":  
    print("Exiting program.")  
    break  
  
else:  
    print("Invalid choice. Try again.")  
  
if __name__ == "__main__":  
    main()
```

Output:

```
S C:\Users\Manav> === Dictionary ADT using Hashing with Chaining ===
> Enter size of the hash table: 5
>
> Choose collision handling mode:
> 1. With replacement (updates existing keys)
> 2. Without replacement (ignores duplicate keys)
> Enter 1 or 2: 1
>
> Options:
> 1. Insert (key, value)
> 2. Find (key)
> 3. Delete (key)
> 4. Display Dictionary
> 5. Exit
> Enter your choice (1-5): 1
> Enter key: Alice
> Enter value: 12345
> Inserted key 'Alice' successfully.
>
> Options:
> 1. Insert (key, value)
> 2. Find (key)
> 3. Delete (key)
> 4. Display Dictionary
> 5. Exit
> Enter your choice (1-5): 1
> Enter key: Bob
> Enter value: 67890
> Inserted key 'Bob' successfully.
>
```